

Prova Finale di Reti Logiche

Anno Accademico 2024 - 2025

Alberto Pini e Davide Redaelli
10823873 - 10899959
212524 - 215263

8 maggio 2026

Indice

1	Introduzione	2
2	Analisi Specifica	2
2.1	Interfaccia del Componente	2
2.2	Struttura Memoria	3
2.3	Computare il Risultato	4
2.4	Funzionamento Modulo	4
2.5	Lavorare con la Memoria	5
2.5.1	Operazione di Scrittura:	5
2.5.2	Operazione di Lettura:	5
2.6	Esempio	5
3	Architettura del Componente	6
3.1	FSM e Descrizione Stati	6
3.2	Segnali e Variabili	7
4	Risultati Sperimentali	9
4.1	Sintesi	9
4.1.1	Analisi del Report di Sintesi	9
4.1.2	Report di Utilizzo delle Risorse	10
4.1.3	Report di Analisi Temporale	12
4.2	TestBench	13
4.2.1	TB standard	13
4.2.2	TB custom filtro singolo	14
4.2.3	Altri TB	14
5	Conclusioni	15

1 Introduzione

Il presente progetto di Reti Logiche ha avuto come obiettivo lo sviluppo di un componente hardware, descritto in VHDL e destinato alla sintesi su FPGA, in grado di implementare le funzionalità richieste dalla specifica.

2 Analisi Specifica

L'FPGA da programmare lavora con una memoria costituita da metadati iniziali e una sequenza di K parole, ciascuna referenziata con W_i . Il Chip dovrà "iterare" la memoria mentre calcola e scrive i risultati.

2.1 Interfaccia del Componente

Diagramma delle porte

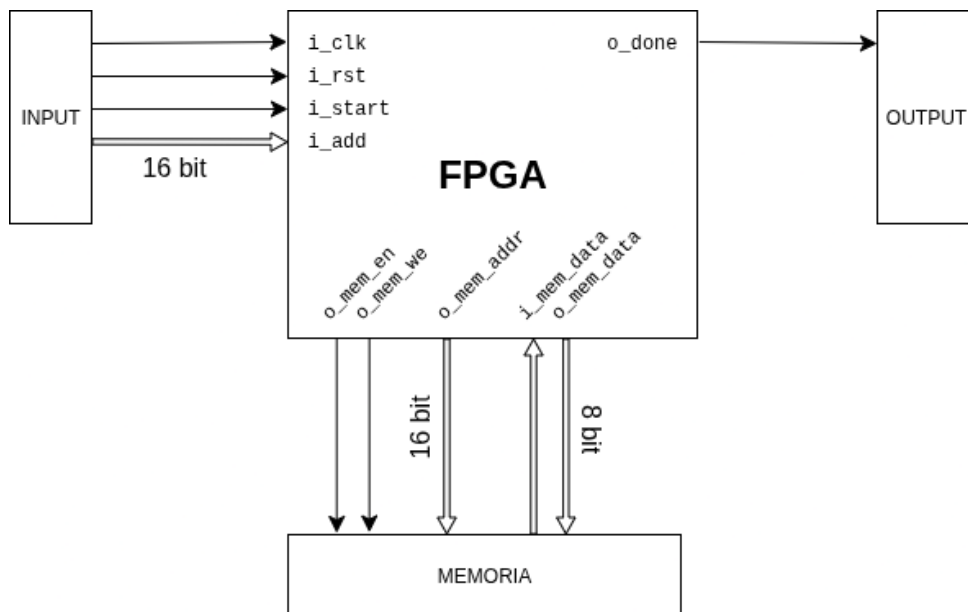


Figura 1: Diagramma delle porte

Notare che nel contesto di development sia INPUT che OUTPUT che MEMORIA fanno tutti parte di un unico chip non sintetizzabile che corrisponde al Testbench.

```
1 entity project_reti_logiche is
2   port (
3       i_clk      : in std_logic;
4       i_rst      : in std_logic;
5       i_start    : in std_logic;
6       i_add      : in std_logic_vector(15 downto 0);
7       o_done     : out std_logic;
8       o_mem_addr : out std_logic_vector(15 downto 0);
9       i_mem_data  : in std_logic_vector(7 downto 0);
10      o_mem_data  : out std_logic_vector(7 downto 0);
11      o_mem_we    : out std_logic;
12      o_mem_en    : out std_logic
13  );
14 end project_reti_logiche;
```

Listing 1: In VHDL l'entità componente viene definita in questo modo

Descrizione delle porte del componente:

- **i_clk** clock di sistema
- **i_rst** segnale di reset asincrono
- **i_start** segnale che indica la fine della scrittura in memoria degli input da parte del testbench e, conseguentemente, la possibilità di iniziare il calcolo da parte del chip
- **i_add** [15 – 0] address della memoria da cui leggere il primo byte di metadati
- **o_done** segnale da alzare una volta che si è finito di scrivere l'ultimo risultato

Le altre porte sono per la lettura e scrittura della memoria

- **o_mem_en** segnale per attivare la memoria sia in lettura che potenzialmente anche in scrittura
- **o_mem_we** segnale per attivare la modalità scrittura della memoria
- **o_mem_addr** [15 – 0] address della memoria su cui il chip vuole lavorare (sia in lettura che scrittura)
- **o_mem_data** [7 – 0] segnale fornito dal chip alla memoria per memorizzare il valore in **o_mem_addr** (con **o_mem_we** = **o_mem_en** = 1)
- **i_mem_data** [7 – 0] segnale fornito al chip dalla memoria corrispondente al valore risultante della precedente lettura (con **o_mem_we** = 0 e **o_mem_en** = 1)

Queste porte sono meglio approfondite nella sezione dedicata 2.5.

2.2 Struttura Memoria

Tutti i dati da utilizzare sono memorizzati sequenzialmente a partire da un indirizzo specificato (ADD). I dati iniziali comprendono:

- **17 byte** che indicano la lunghezza della sequenza, l'ordine del filtro differenziale e i coefficienti.
 - **K1** e **K2**: due byte che indicano la lunghezza della sequenza K dei dati di W (K1 è il byte più significativo, lunghezza minima = 7 byte).
 - **S**: un byte che indica quale filtro selezionare tra ordine 3 e ordine 5 (bit meno significativo: 0 → ordine 3, 1 → ordine 5).
 - Da **C1** a **C14**: 14 byte contenenti i coefficienti dei filtri. I coefficienti sono presenti per entrambi i filtri indipendentemente dal valore di S. Per esempio
 - * Ordine 3: $c=[0, -1, 8, 0, -8, 1, 0]$ da C1 a C7, normalizzazione $n = 12$.
 - * Ordine 5: $c=[1, -9, 45, 0, -45, 9, -1]$ da C8 a C14, normalizzazione $n = 60$.
- **K byte** da elaborare, contenenti valori tra -128 e +127, che rappresentano il vettore numerico da processare.

Graficamente:

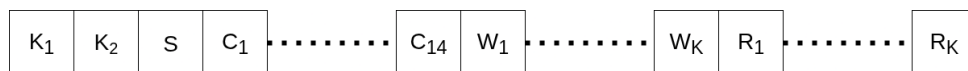


Figura 2: Struttura della memoria

Quindi unendo gli 8 bit di K_1 con gli altri 8 bit di K_2 ottengo un numero K di 16 bit. Questo numero è fondamentale per:

- capire quando terminare la lettura delle W_i da processare (quando si arriva a $17 + K$ byte)
- saper in che indirizzo di memoria scrivere il primo risultato R_1 (al $(17 + K + 1)$ esimo byte)

2.3 Computare il Risultato

Il filtro applicato ha la seguente formula matematica:

$$f'(i) = \frac{1}{n} \sum_{j=-I}^I C_j \cdot f[j+i]$$

Dove **I=2** per il filtro di ordine 3 e **I=3** per il filtro di ordine 5.

Bisogna inoltre assicurarsi che il risultato del filtraggio prima della normalizzazione ($\sum C_j \cdot f[j+i]$) abbia un numero di bit adeguato per la sua rappresentazione.

La divisione per **n** viene approssimata come segue:

- $\frac{1}{12} \approx \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \frac{1}{1024}$
- $\frac{1}{60} \approx \frac{1}{64} + \frac{1}{1024}$

Poiché ogni shift a destra equivale a una divisione per due, nel caso di numeri negativi si applica una correzione: se $\text{val} \gg m < 0$, si aggiunge +1 per ridurre l'errore di troncamento.

Per esempio:

$$-40 \times \frac{1}{12} \approx (-40 \gg 4 + 1) + (-40 \gg 6 + 1) + (-40 \gg 8 + 1) + (-40 \gg 10 + 1)$$

2.4 Funzionamento Modulo

Il modulo ha i seguenti segnali:

- Ingressi:
 - **i_start** = **START** (1 bit): avvia l'elaborazione.
 - **i_add** (16 bit): indirizzo della memoria di partenza.
 - **i_clk** = **CLOCK**
 - **i_rst** = **RESET**.
- Uscite:
 - **o_done** = **DONE** (1 bit): indica la fine dell'elaborazione al testbench.

Il modulo segue la seguente sequenza operativa:

1. All'avvio **RESET=1** e **DONE=0**.
2. Quando **RESET** torna a 0, il modulo attende che, quando il **CLOCK** sale ad 1, **START** sia pari a 1.
3. Legge i dati da memoria, esegue il filtraggio e scrive i risultati.
4. Alla fine dell'elaborazione, alza **DONE=1** per segnalare al testbench che il modulo ha finito il suo lavoro.
5. Il modulo attende che **START** torni a 0 prima di una nuova fase di computazione.
6. Infine il modulo setta **DONE=0** per segnalare al testbench che è pronto a ricevere un nuovo **START=1**.

In qualsiasi momento **RESET** può passare da $0 \rightarrow 1$ e, in modo asincrono (quindi indipendentemente dal segnale **i_clk**), il componente deve essere in grado di:

1. fermare l'esecuzione, settando gli stati appropriati
2. inizializzare tutti i segnali interni, specialmente i contatori

2.5 Lavorare con la Memoria

La comunicazione con la memoria esterna, gestita a livello del Top Module, avviene tramite un'interfaccia standardizzata che include i seguenti connettori:

- **o_mem_en** (Output, `std_logic`): Questo segnale viene asserito a 1 per abilitare la comunicazione con la memoria esterna, sia per operazioni di lettura che di scrittura. Quando **o_mem_en** è 0, la memoria è disabilitata e non risponde alle richieste.
- **o_mem_we** (Output, `std_logic`): Questo segnale definisce il tipo di operazione sulla memoria. Se **o_mem_we** è 0, l'operazione è di lettura. Se **o_mem_we** è 1, l'operazione è di scrittura. È fondamentale che **o_mem_en** sia attivo quando **o_mem_we** viene pilotato.
- **o_mem_addr** (Output, `std_logic_vector(15 downto 0)`): Questo è l'indirizzo a 16 bit nella memoria su cui il sistema desidera operare. Viene utilizzato sia per le letture che per le scritture, specificando l'indirizzo di memoria target (in cui scrivere o da cui leggere).
- **o_mem_data** (Output, `std_logic_vector(7 downto 0)`): Questa porta serve ad "inviare" il dato a 8 bit che il componente intende scrivere nella memoria. Questo segnale è valido e viene letto dalla memoria solo quando **o_mem_en**=1 e **o_mem_we** = 1.
- **i_mem_data** (Input, `std_logic_vector(7 downto 0)`): Questo connettore riceve il dato a 8 bit letto dalla memoria esterna. Il valore su **i_mem_data** è valido e deve essere campionato dal componente quando **o_mem_en**=1 e **o_mem_we**=0, e dopo il tempo di propagazione della memoria.

Data la mancanza di un segnale di acknowledgment da parte della memoria, consideriamo l'operazione conclusa sempre 1 ciclo di clock dopo aver mandato il comando.

2.5.1 Operazione di Scrittura:

1. Al fronte di salita del clock T_0 , il sistema asserisce **o_mem_en** = 1, **o_mem_we** = 1, **o_mem_addr** all'indirizzo desiderato e **o_mem_data** con il valore da scrivere.
2. Al fronte di salita del clock T_1 (cioè 1 CLK dopo), la scrittura nella memoria è considerata completata e il dato è stato memorizzato all'indirizzo specificato. Il sistema può quindi modificare i due segnali di controllo oppure avviare una nuova operazione.

2.5.2 Operazione di Lettura:

1. Al fronte di salita del clock T_0 , il sistema asserisce **o_mem_en** = 1, **o_mem_we** = 0 e **o_mem_addr** all'indirizzo da leggere.
2. Al fronte di salita del clock T_1 (cioè 1 CLK dopo), il dato letto dalla memoria sarà disponibile e stabile sul connettore **i_mem_data** e potrà essere campionato dal sistema. Il sistema può quindi disasserire i segnali di controllo o avviare una nuova operazione.

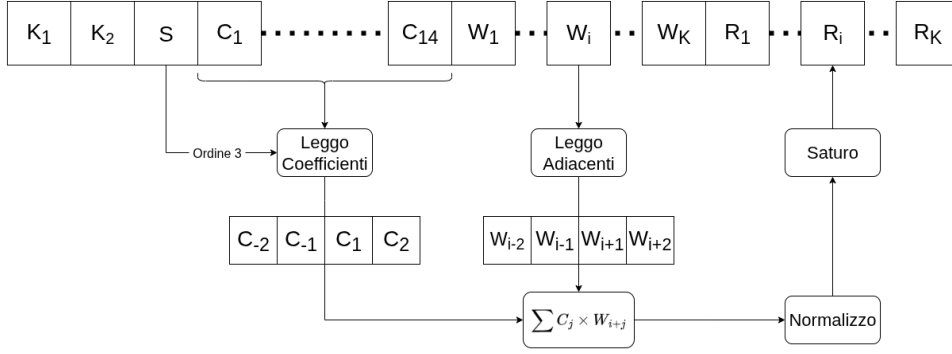
2.6 Esempio

Per illustrare il processo di filtraggio differenziale, consideriamo l'applicazione di un filtro di ordine 3 a una sequenza generica di dati. Si assume che il modulo abbia già letto dalla memoria i parametri iniziali, ovvero la lunghezza della sequenza K , il bit di selezione del filtro S (che indica l'uso di un filtro di ordine 3), e i coefficienti del filtro. La sequenza dei coefficienti sarà composta da $C_{-3}, C_{-2}, C_{-1}, C_0, C_1, C_2, C_3$ (letti in base all'ordine del filtro) e il valore di normalizzazione n . L'indirizzo di inizio dei dati in input è ADD_W , e l'indirizzo dove verranno scritti i risultati è ADD_R .

Il processo iterativo per il calcolo di ciascun valore filtrato R_i è il seguente:

1. **Lettura dei dati di input:** Per calcolare il valore R_i , il modulo deve accedere ai valori W circostanti la posizione i nella memoria. Nello specifico, per un filtro di ordine 3, saranno necessari i valori $W_{i-2}, W_{i-1}, W_i, W_{i+1}, W_{i+2}$. Questi vengono letti dagli indirizzi di memoria appropriati, che sono $ADD_W + (i - 2)$, $ADD_W + (i - 1)$, $ADD_W + i$, $ADD_W + (i + 1)$, $ADD_W + (i + 2)$. Si noti che per i valori all'inizio o alla fine della sequenza W (cioè quando gli indici $i - 2$, $i - 1$, i , $i + 1$, $i + 2$ eccedono i limiti della sequenza valida), i valori corrispondenti a W non esistenti sono considerati pari a 0.

Figura 3: Diagramma di esempio di funzionamento



2. **Calcolo della somma pesata:** Viene calcolata la somma pesata dei valori di input con i rispettivi coefficienti del filtro. La formula generale per un filtro differenziale è

$$R_i^{\text{raw}} = \sum_{j=-l}^{+l} C_j \times W_{i+j} \quad \text{dove } l = \begin{cases} 2 & \text{per il filtro di ordine 3} \\ 3 & \text{per il filtro di ordine 5} \end{cases}$$

Pertanto, il valore intermedio (di ordine 3) sarà:

$$R_i^{\text{raw}} = (C_{-2} \times W_{i-2}) + (C_{-1} \times W_{i-1}) + (C_0 \times W_i) + (C_1 \times W_{i+1}) + (C_2 \times W_{i+2})$$

Si assume che i coefficienti C_j siano quelli specificati per il filtro di ordine 3, come ad esempio i coefficienti $C_{-2}, C_{-1}, C_0, C_1, C_2$ dalla specifica $c = [-1, 8, 0, -8, 1]$. Se invece stessimo filtrando in ordine 5 i C_j sarebbero stati 7 in totale ($C_{-3}, C_{-2}, C_{-1}, C_0, C_1, C_2, C_3$).

3. **Normalizzazione:** Il valore R_i^{raw} ottenuto deve essere normalizzato dividendolo per il coefficiente n . Questa divisione è approssimata come una somma di shift a destra (divisioni per potenze di 2). Per il filtro di ordine 3, $n = 12$, approssimato come $1/16 + 1/64 + 1/256 + 1/1024$. Per minimizzare l'errore di troncamento per i numeri negativi, viene aggiunto 1 al risultato di un singolo shift se il valore shiftato è negativo.
4. **Saturazione:** Dopo la normalizzazione, il valore R_i risultante viene confrontato con l'intervallo valido di $[-128, +127]$. Se R_i è inferiore a -128 , viene saturato a -128 . Se R_i è superiore a $+127$, viene saturato a $+127$.
5. **Scrittura in memoria:** Il valore finale R_i viene scritto in memoria all'indirizzo $\text{ADD}_R + i$.

Questo ciclo si ripete per tutti i K valori della sequenza, dopodiché il modulo segnala il completamento dell'operazione.

3 Architettura del Componente

Per questo progetto abbiamo creato un unico modulo (`project_reti_logiche`) che agisce ovviamente da `top_module`. Al suo interno è presente un unico processo sequenziale contenente tutta la logica dell'FPGA, come appunto si vedrà in seguito.

3.1 FSM e Descrizione Stati

Ogni transizione da uno stato ad un altro nella macchina a stati finiti, include una transizione intermedia ad uno stato di "attesa", necessario per permettere ai testbench di interagire con la memoria e di inviare al successivo ciclo di clock i corretti segnali richiesti dal modulo.

Prima della transizione allo stato WAITING, sarà computato e modificato il segnale `next_state`, che rappresenta dunque il prossimo stato. Figura 2

Inoltre il segnale `i_rst` agisce come segnale asincrono. Ciò vuol dire che, indipendentemente dal segnale di clock, l'attivazione di `i_rst` porta a:

- reset di ogni segnale interno
- settaggio di `state` $\leq$ WAITING e di `next_state` $\leq$ START

Di seguito i diagrammi del cambio di stato (Figura 4) e della FSM principale (Figura 5).

3.2 Segnali e Variabili

Questi segnali sono utilizzati per la gestione interna dei dati e il controllo del flusso all'interno del modulo:

- **Segnali per gli stati:**
 - `next_state`: Segnale di tipo `state_type` (stati descritti precedentemente) che memorizza lo stato futuro a cui la FSM transirà nel prossimo ciclo di clock.
 - `state`: Segnale di tipo `state_type` che rappresenta lo stato corrente in cui si trova la FSM.
- **Contatori interni:**
 - `data_counter`: Contatore intero che tiene traccia del numero di parole W elaborate e funge da indice per l'accesso ai dati in input e la scrittura dei risultati in output.
 - `coeff_counter`: Contatore intero che tiene traccia del numero di coefficienti del filtro letti durante la fase di inizializzazione.
- **Segnali per memorizzare i coefficienti:**
 - `c_n2_3`, `c_n1_3`, `c_central_3`, `c_p1_3`, `c_p2_3`: Segnali a 8 bit (**signed**) che memorizzano i coefficienti specifici per il filtro di ordine 3. Corrispondono ai coefficienti $C_{-2}, C_{-1}, C_0, C_{+1}, C_{+2}$ dalla specifica, assumendo C_0 sia un valore centrale e altri coefficienti siano impliciti o zero.
 - `c_n3_5`, `c_n2_5`, `c_n1_5`, `c_central_5`, `c_p1_5`, `c_p2_5`, `c_p3_5`: Segnali a 8 bit (**signed**) idem a prima ma per il filtro di ordine 5.
- **Segnali della "sliding window":**
 - `prev3`, `prev2`, `prev1`: Segnali a 8 bit (**signed**) che contengono i valori di input W precedenti all'attuale valore `current_W` nel buffer di scorrimento.
 - `current_W`: Segnale a 8 bit (**signed**) che contiene il valore di input W attualmente in elaborazione.
 - `next1`, `next2`, `next3`: Segnali a 8 bit (**signed**) che contengono i valori di input W successivi all'attuale valore `current_W` nel buffer di scorrimento.

Questi valori alla fine di ogni calcolo vengono shiftati per far spazio al nuovo valore letto in `next2` o `next3` a seconda dell'ordine del filtro.

- **Altri segnali interni:**
 - `mem_w1_addr`: Indirizzo a 16 bit che punta all'inizio della sequenza di dati W da filtrare in memoria. È calcolato sommando l'indirizzo base `i_addr` e l'offset dei metadati iniziali (ovvero $17 = 3$ byte per K e $S + 14$ byte per i coefficienti).
 - `mem_init_addr`: Indirizzo a 16 bit che memorizza l'indirizzo di partenza `i_addr` fornito all'inizio dell'esecuzione, usato come base per accedere ai metadati.
 - `k1`, `k2`: Segnali a 8 bit che memorizzano i due byte letti dalla memoria che insieme formano la lunghezza K della sequenza di input.
 - `k`: Segnale a 16 bit (**unsigned**) che memorizza la lunghezza effettiva della sequenza di dati W da elaborare.
 - `s`: Segnale a 1 bit (**std_logic**) che indica il tipo di filtro: 0 per l'ordine 3, 1 per l'ordine 5.

Ci sono poi le variabili che vengono utilizzate temporaneamente all'interno del processo sequenziale per calcoli intermedi e non mantengono il loro valore tra un ciclo di clock e l'altro:

Figura 4: Generico cambio di stato con segnale di reset

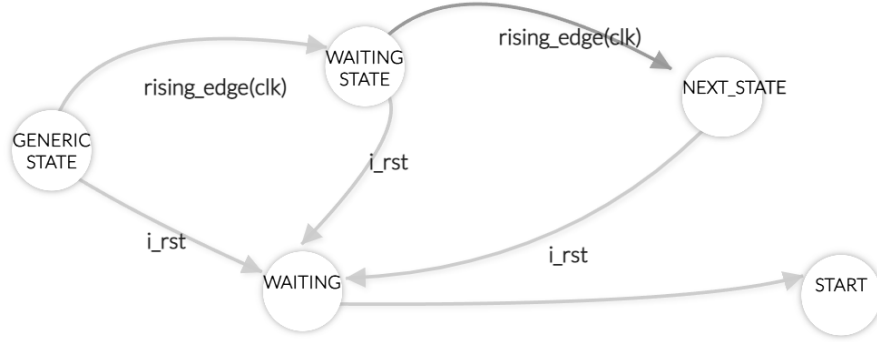
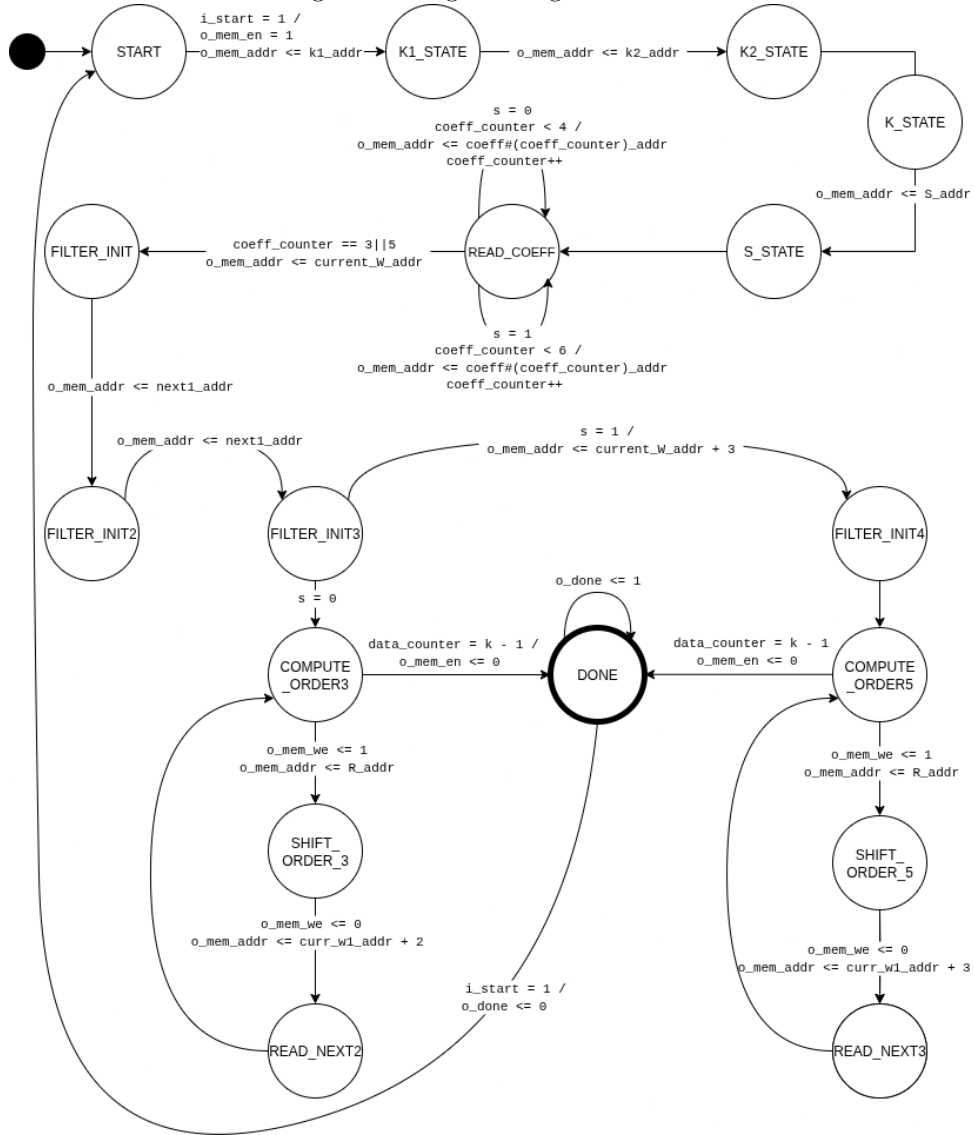


Figura 5: Diagramma generale FMS



- `tmp_k`: Variabile a 16 bit (`std_logic_vector`) utilizzata per concatenare `k1` e `k2` prima di assegnare il risultato a `k`.
- `tmp_s`: Variabile a 1 bit (`std_logic`) utilizzata per una copia temporanea del bit di selezione del filtro `s` per il controllo immediato del flusso.
- `tmp_p3`, `tmp_p2`, `tmp_p1`, `tmp_central`, `tmp_n1`, `tmp_n2`, `tmp_n3`: Variabili a 32 bit (`signed`) utilizzate per memorizzare i prodotti intermedi dei coefficienti per i valori di input. Vengono utilizzate con una larghezza maggiore per prevenire overflow durante le moltiplicazioni.
- `tmp_sum`: Variabile a 32 bit (`signed`) che accumula la somma totale dei prodotti parziali tri coefficienti e i valori di input.
- `tmp_res`: Variabile a 32 bit (`signed`) che contiene il risultato del filtraggio dopo la normalizzazione, prima dell'applicazione della saturazione e della conversione a 8 bit.

In particolare vengono utilizzate in questo modo nello stato di `COMPUTE_ORDER_3`

```

1 tmp_n2 := resize(c_n2_3, 16) * resize(prev2, 16);
2 tmp_n1 := resize(c_n1_3, 16) * resize(prev1, 16);
3 tmp_central := resize(c_central_3, 16) * resize(current_W, 16);
4 tmp_p1 := resize(c_p1_3, 16) * resize(next1, 16);
5 tmp_p2 := resize(c_p2_3, 16) * resize(next2, 16);
6
7 tmp_sum := tmp_n2 + tmp_n1 + tmp_central + tmp_p1 + tmp_p2;
8 tmp_res := shift_right (tmp_sum, 4) + shift_right (tmp_sum, 6)
9           + shift_right (tmp_sum, 8) + shift_right (tmp_sum, 10);
10 if tmp_sum < to_signed(0, 32) then
11     tmp_res := tmp_res + 4;
12 end if;
```

Listing 2: Calcolo mediante variabili temporanee

4 Risultati Sperimentali

4.1 Sintesi

La sintesi è il processo che traduce la descrizione hardware (il programma VHDL in questo caso) in un elenco di come i blocchi base dell'FPGA (le 'primitive') devono essere collegati tra loro.

4.1.1 Analisi del Report di Sintesi

Il processo di sintesi è stato eseguito con successo, come indicato dalle linee finali del report:

```

1 Synthesis finished with 0 errors, 0 critical warnings and 0 warnings.
2 synth_design completed successfully
```

Listing 3: Messaggi di successo della sintesi

Questo conferma che la descrizione hardware è sintatticamente e semanticamente corretta per la sintesi e che non ci sono stati problemi bloccanti durante il processo.

Sintesi eseguita con *Vivado v2015.4 (64-bit)* su un sistema Linux. E il dispositivo FPGA di destinazione è stato specificato come `xc7k70tfbg676-1`.

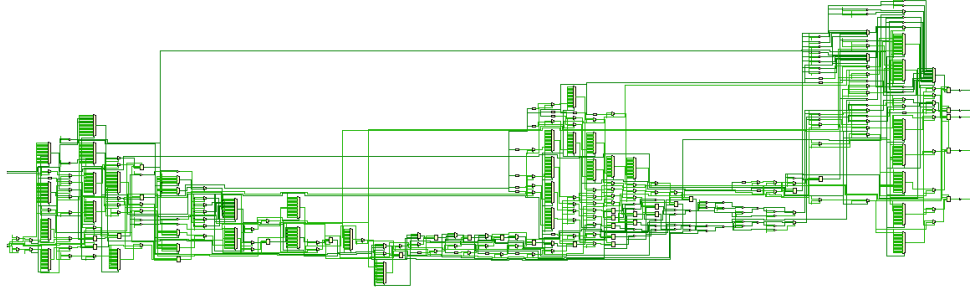
Nel report è presente un avviso significativo:

```

1 WARNING: [Netlist 29-101] Netlist 'project_reti_logiche' is not ideal for
    floorplanning, since the cellview 'project_reti_logiche' contains a large
    number of primitives. Please consider enabling hierarchy in synthesis if you
    want to do floorplanning.
```

Questo warning suggerisce che il design, pur essendo funzionalmente corretto, potrebbe non essere ottimale per il **floorplanning** (la disposizione fisica dei blocchi sul chip) a causa di un'elevata "piattezza" (flattening). Visivamente l'immagine dello schema delle primitive mostra proprio questa mancanza di astrazione e gerarchia:

Figura 6: Schematic del componente



Inoltre, sono presenti diversi avvisi relativi al mancato mappaggio di alcune ROM (Read-Only Memory) a blocchi di RAM dedicati ('BRAM') a causa di una dimensione dell'indirizzo superiore al massimo supportato (32 bit vs 25 bit). Questo implica che le ROM in questione sono state implementate utilizzando logica combinatoria (LUTs) invece di risorse BRAM più efficienti, il che potrebbe incidere sull'utilizzo delle risorse logiche e sulle performance.

```
INFO: [Synth 8-5545] ROM "o_mem_addr" won't be mapped to RAM because address
size (32) is larger than maximum supported (25)
```

Questo è ripetuto per diverse istanze di ROM (o_mem_addr, c_n2_3, c_n1_3, c_p1_3, c_p2_3, c_n3_5, c_n2_5, c_n1_5, c_p1_5, c_p2_5, c_p3_5).

4.1.2 Report di Utilizzo delle Risorse

Il report utilization fornisce una panoramica dettagliata delle risorse logiche e fisiche usate dall'FPGA dopo la sintesi.

La Tabella 1 riassume l'utilizzo delle risorse fondamentali all'interno delle Slice Logiche (quindi i blocchetti logici configurabili all'interno dell'FPGA).

Tabella 1: Riepilogo Utilizzo Logica Slice

Tipo di Risorsa	Utilizzate	Disponibili	Util%
Slice LUTs	1608	41000	3.92%
LUT as Logic	1608	41000	3.92%
LUT as Memory	0	13400	0.00%
Slice Registers	302	82000	0.37%
Register as Flip Flop	302	82000	0.37%
Register as Latch	0	82000	0.00%
F7 Muxes	31	20500	0.15%
F8 Muxes	0	10250	0.00%

Da notare che il componente vero e proprio utilizza una percentuale molto bassa delle risorse logiche di base del chip FPGA (meno del 4% di LUTs e meno dell'1% di registri).

La distribuzione dei registri per tipo (sincroni/asincroni, con/senza Clock Enable, Set/Reset) è fornita nella Tabella 2.

Tabella 2: Distribuzione dei Registri per Tipo

Totale	Clock Enable	Synchronous	Asynchronous	Descrizione
3	Yes	-	Set	Registri con CE e Set Asincrono
115	Yes	-	Reset	Registri con CE e Reset Asincrono
184	Yes	Reset	-	Registri con CE e Reset Sincrono
302	Totale Registri Utilizzati			

Tabella 3: Utilizzo Risorse di Memoria (BRAM)

Tipo di Memoria	Utilizzate	Disponibili	Util%
Block RAM Tile	0	135	0.00%
RAMB36/FIFO*	0	135	0.00%
RAMB18	0	270	0.00%

L'assenza di utilizzo di BRAM è in linea con i warning di sintesi relativi al mancato mappaggio delle ROM a causa delle dimensioni dell'indirizzo.

Tabella 4: Utilizzo Risorse di I/O

Tipo di Risorsa I/O	Utilizzate	Disponibili	Util%
Bonded IOB	54	300	18.00%

Tabella 5: Utilizzo Risorse di Clocking

Tipo di Risorsa Clocking	Utilizzate	Disponibili	Util%
BUFGCTRL	1	32	3.13%

I 54 pin di I/O utilizzati rappresentano una percentuale moderata dei pin disponibili (18%). È utilizzato un solo Global Buffer per la gestione del clock.

La Tabella 6 ripresenta un riepilogo delle primitive elementari effettivamente istanziate nel design dopo la sintesi, come già visto nel report di sintesi, con il conteggio specifico per ciascun tipo.

Tabella 6: Riepilogo delle Primitive Utilizzate

Nome Primitiva	Categoria Funzionale	Conteggio
LUT6	LUT	461
LUT2	LUT	460
LUT1	LUT	382
LUT4	LUT	352
CARRY4	CarryLogic	306
LUT3	LUT	205
FDRE	Flop & Latch	184
LUT5	LUT	175
FDCE	Flop & Latch	115
MUXF7	MuxFx	31
OBUF	IO	27
IBUF	IO	27
FDPE	Flop & Latch	3
BUFG	Clock	1
Totale LUTs		2035
Totale Flip-Flop		302

Il design è stato implementato con un utilizzo relativamente basso delle risorse logiche programmabili della FPGA (LUTs e Flip-Flop).

In sintesi, i risultati di utilizzo principali sono riportati dalla tabella 7

Tabella 7: Riepilogo Utilizzo delle Risorse della FPGA

Risorsa	Utilizzo	Disponibile	Utilizzo %
LUT	1608	41000	3.92
FF	302	82000	0.37
IO	54	300	18.00
BUFG	1	32	3.12

4.1.3 Report di Analisi Temporale

Il report (nella tabella sottostante) fornisce un'indicazione delle prestazioni del design in termini di tempi di setup, hold e larghezza di impulso. I valori riportati sono i seguenti:

Tabella 8: Riepilogo dei Risultati dell'Analisi Temporale

Categoria	Parametro	Valore
Setup	Worst Negative Slack (WNS)	7.804 ns
	Total Negative Slack (TNS)	0.000 ns
	Number of Failing Endpoints	0
	Total Number of Endpoints	528
Hold	Worst Hold Slack (WHS)	0.074 ns
	Total Hold Slack (THS)	0.000 ns
	Number of Failing Endpoints	0
	Total Number of Endpoints	528
Pulse Width	Worst Pulse Width Slack (WPWS)	9.650 ns
	Total Pulse Width Negative Slack (TPWS)	0.000 ns
	Number of Failing Endpoints	0
	Total Number of Endpoints	303

Di seguito una descrizione delle voci della tabella:

- **Setup Slack (WNS - Worst Negative Slack):** È il margine di tempo minimo in cui i dati devono essere stabili e validi all'ingresso di un flip-flop prima che arrivi il fronte di clock attivo. In sostanza indica che i dati sono stabili nel FF almeno x ns prima del fronte di salita del clock.
- **Hold Slack (WHS - Worst Hold Slack):** È il margine di tempo minimo in cui i dati devono rimanere stabili all'ingresso di un flip-flop dopo l'arrivo del fronte di clock attivo. Anche qui, un valore positivo indica conformità, un valore negativo indica una violazione.
- **Pulse Width Slack (WPWS - Worst Pulse Width Slack):** Questo parametro misura il tempo aggiuntivo che la durata di un impulso (sia esso un segnale di clock o un segnale di controllo come reset o enable) ha a disposizione, rispetto al periodo minimo richiesto, per garantire il corretto funzionamento dei flip-flop e della logica sequenziale.
- **Total Negative Slack (TNS / THS / TPWS):** Rappresenta la somma di tutte le violazioni di temporizzazione (slack negativi) all'interno di una specifica categoria (Setup, Hold, Pulse Width).
- **Number of Failing Endpoints:** Indica il numero di punti (ad esempio, registri o pin di output) nel circuito dove è stata rilevata una violazione di temporizzazione (setup, hold o pulse width).
- **Total Number of Endpoints:** Indica il numero totale di punti nel design che vengono analizzati per i requisiti di temporizzazione in una specifica categoria.

I dati presentati nella Tabella 8 mostrano una chiara indicazione della salute temporale del design:

- **Per la categoria Setup:**
 - Il WNS di 7.804 ns indica un ampio margine di setup, ben oltre il minimo richiesto. Questo significa che i dati arrivano con notevole anticipo rispetto al fronte di clock, garantendo un funzionamento sicuro.

- Il TNS di 0.000 ns e un Number of Failing Endpoints pari a 0 confermano che non ci sono assolutamente violazioni di setup in nessun percorso del design.

- **Per la categoria Hold:**

- Il WHS di 0.074 ns è un valore positivo, seppur piccolo, che indica che anche i requisiti di hold sono soddisfatti.
- Il THS di 0.000 ns idem a prima.

- **Per la categoria Pulse Width:**

- Il WPWS di 9.650 ns indica un ampio margine positivo anche per la larghezza degli impulsi.
- Il TPWS di 0.000 ns idem a prima.

4.2 TestBench

Il modulo è stato sottoposto ai seguenti TestBench, superandoli tutti fino alla fase di simulazione post sintesi functional. Per il testbench 2345 è stata effettuata anche la timing.

Tabella 9: Riepilogo dei Risultati delle Simulazioni dei Testbench

Nome Testbench	Behavioral	Functional
Default		
tb2425.vhd	OK	OK
Generati con filtro singolo		
tb_3_1500.vhd	OK	OK
tb_3_32000.vhd	OK	OK
tb_5_24.vhd	OK	OK
tb_5_1500.vhd	OK	OK
tb_5_32000.vhd	OK	OK
Corner Cases		
TB.Intermedi_Negativi.vhdl	OK	OK
TB.Intermedi_Positivi.vhdl	OK	OK
tb_ordine_3_con_C1_e_C7_non_0.vhd	OK	OK
Altri		
tb_new.vhd	OK	OK
TestBench_mark2.vhdl	OK	OK

I testbench a cui si fa riferimento nella tabella sono contenuti nella directory **TestBench** nella cartella root della repo.

4.2.1 TB standard

Il testbench standard **tb_2425.vhd** è stato il primo utilizzato per la verifica del design. Il suo superamento indica la corretta implementazione di diversi aspetti fondamentali del componente, quali:

- La corretta gestione della memoria (vedi figura 9).
- Il calcolo del risultato con un filtro di ordine 3.
- La generazione e l'interpretazione dei segnali in accordo con il testbench (come mostrato nella figura 7 e 8).

Figura 7: $i_start:0 \rightarrow 1$ e la modifica degli stati

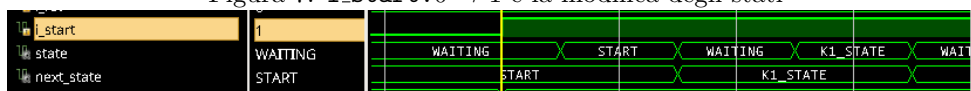


Figura 8: `o_done:0 → 1`, l'abbassamento di `i_start` e la modifica degli stati

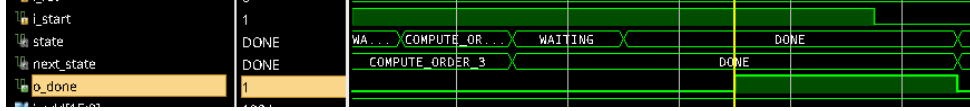
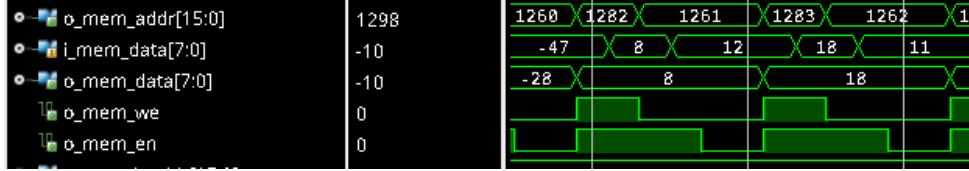


Figura 9: scrittura di un dato sulla memoria



Questo test ha avuto lo scopo di convalidare le funzionalità essenziali del componente. Il successo di questa verifica funge da base per successivi affinamenti del codice, volti alla gestione dei casi limite (cfr. 4.2.2) o all'ottimizzazione dell'efficienza in previsione di testbench più onerosi.

4.2.2 TB custom filtro singolo

- **tb_3_1500, tb_3_32000:** I test effettuati con queste batterie sono stati utili a verificare la capacità del modulo di gestire anche input di dimensioni più grandi (rispettivamente 1500 e 32000 parole *W* da filtrare) rispetto al `tb_2345`, sempre con il filtro di ordine 3.
- **tb_5_24, tb_5_1500, tb_5_32000:** Questi testbenches hanno testato la funzione di filtraggio di ordine 5, sia con piccole sequenze in input (`tb_5_24`), sia con lunghe sequenze (32000 parole).
- **tb_ordine_3_con_C1_e_C7_non_0:** Batterie che hanno verificato che il filtro di ordine 3 utilizzasse i corretti valori di input, ignorando correttamente valori non nulli ma inutili forniti in input all'interno della sequenza di coefficienti *C*.

I testbench sopra citati con 1500 e 32000 elementi sono stati generati automaticamente dal sito Testbench-Generator

4.2.3 Altri TB

Questa sezione descrive ulteriori testbench sviluppati per la verifica del componente, focalizzandosi su scenari specifici e casi limite non coperti dai test di base.

- **tb_new.vhdl** Questo testbench è stato progettato per effettuare una verifica completa del componente sotto diversi scenari di input e per testare specifiche funzionalità avanzate. Gli obiettivi principali di questo testbench includono:
 - La verifica del componente con diverse sequenze di input.
 - Il controllo del corretto troncamento del risultato (saturazione).
 - La validazione del funzionamento del componente anche dopo aver ricevuto un segnale di reset durante un'elaborazione in corso.
 - Il testing standard sia del filtro di terzo ordine che di quello di quinto ordine.
 - L'assicurazione che il filtro di terzo ordine operi correttamente anche quando i coefficienti iniziali e finali non sono impostati a zero.
 - La conferma che il componente possa avviare una nuova esecuzione senza la necessità di un segnale di reset aggiuntivo.

Il testbench `tb_new.vhdl` realizza questi obiettivi utilizzando e confrontando i dati di esempio (definiti internamente come 'scenario.input/output' e 'scenario.input2/output2'), che vengono testati in successione. Il test include anche la manipolazione del segnale 'tb_rst' a metà di uno scenario per simulare un reset durante l'operazione.

- **TestBench_mark2.vhdl** Questo testbench è stato progettato per una verifica approfondita del componente, coprendo un'ampia gamma di scenari operativi e condizioni critiche. Si distingue per la sua complessità e il numero di scenari integrati, che includono:
 - Verifica del componente sotto diversi scenari di input (6 per la precisione) complessi e di grandi dimensioni (es. `SCENARIO_LENGTH` fino a 12000 campioni), simulando lunghe elaborazioni.
 - Solito test di saturazione del risultato.
 - Prova della funzionalità del componente dopo aver ricevuto segnali di reset multipli (`tb_rst`) in momenti diversi durante un'elaborazione in corso. Questo verifica la robustezza e la capacità di ripristino del design.
 - Testing sia del filtro di terzo ordine che di quello di quinto ordine, applicando diverse configurazioni e set di coefficienti.
 - Assicurazione del funzionamento corretto del filtro di terzo ordine anche quando i coefficienti iniziali e finali non sono impostati a zero.
 - Verifica della capacità del componente di avviare nuovamente l'esecuzione senza la necessità di un segnale di reset aggiuntivo tra le diverse esecuzioni.
 - Inclusione di scenari specifici per condizioni limite ('Scenario 5' con tutti 127 e 'Scenario 6' con tutti -128 tra i dati di input) per testare il comportamento del sistema ai massimi e minimi valori rappresentabili.

Il testbench esegue 6 scenari di test in successione, ognuno con proprie configurazioni (`scenario_config`), dati di input (`scenario_inputX`) e output attesi (`scenario_outputX`), pre-caricando i dati nella RAM simulata e verificando i risultati. La logica di controllo gestisce lo scambio della memoria tra il testbench e il componente, e simula reset in punti strategici per una verifica esaustiva della resilienza.

5 Conclusioni

Il progetto presentato in questa relazione ha raggiunto con successo l'obiettivo di sviluppare un componente VHDL per il filtraggio differenziale di sequenze di dati, rispettando pienamente le specifiche fornite. Il modulo implementato gestisce correttamente l'interfacciamento con la memoria esterna, l'interpretazione dei metadati e l'esecuzione di filtri di ordine 3 e 5, producendo i risultati attesi.

La validità del componente è stata confermata attraverso un'approfondita campagna di verifica. Oltre al superamento del testbench standard, sono stati sviluppati e superati numerosi test customizzati, mirati a esplorare un'ampia gamma di scenari operativi. Tale approccio metodico alla verifica ha permesso di consolidare la fiducia nella correttezza e resilienza del design.

L'analisi dei report di sintesi e di temporizzazione ha confermato la fattibilità del progetto sull'hardware target. L'utilizzo delle risorse della FPGA `xc7k70tfbg676-1` è risultato contenuto, e l'assenza di violazioni di timing assicura il funzionamento affidabile del componente alla frequenza di clock richiesta.

In conclusione, lo sviluppo di questo progetto ha rappresentato un'importante opportunità formativa, consolidando le conoscenze teoriche di progettazione digitale e offrendo un'esperienza pratica completa che spazia dalla scrittura del codice VHDL alla sua verifica funzionale, sintesi e analisi delle performance.